

Guess the Mystery Word

Project Report Submission

Submitted by: Yash Dwivedi

In fulfillment of the Build Your Own Project requirement

VITyarthi Flipped Course Evaluation - CSE (Problem Solving)

1. Introduction

This document is the project report for "Guess the Mystery Word", a terminal-based Python application developed to test logical deduction through progressive riddles. The project was built to align with the technical expectations of the CSE (Problem Solving) flipped course evaluation [cite: 1, 7].

2. Problem Statement

Students require lightweight, terminal-based applications that foster problem-solving and logical deduction without the overhead of complex graphical interfaces. This project addresses this by offering a procedurally generated console game that tests both technical vocabulary and deductive reasoning.

3. Functional Requirements

The system must include the following capabilities [cite: 1]:

- **Level Progression:** The game must advance the user through a 2D list of levels upon successful guesses.
- **Attempt Tracking:** The system must deduct attempts for incorrect guesses and terminate the game when attempts reach zero.
- **Input Validation:** The program must normalize user input using the `.capitalize()` method to ensure accurate string comparison.
- **Terminal UI:** The application must display formatted text (bold, italic, centered) to guide the user.

4. Non-Functional Requirements

- **Performance:** The lightweight script must execute instantaneously in any standard terminal.
- **Usability:** The console output must clearly distinguish between hints, warnings, and success messages using structural centering and ANSI escape sequences.
- **Reliability:** The while-loop conditions must prevent infinite loops regardless of user input.
- **Maintainability:** The procedural design allows for easy addition of new levels by simply appending to the `levels` list.

5. System Architecture

The application employs a procedural programming paradigm relying on sequential execution and nested loop structures.

- **Data Structure:** A 2D list (`levels`) stores the target word, hint, and maximum attempts for each stage.
- **Control Flow:** An outer `while` loop manages the overall level progression, while an inner `while` loop handles the user attempt tracking for the current level.

6. Design Diagrams

Workflow Diagram

[Workflow: Start -> Print Level & Hint -> Accept Input -> Check Input == Target? -> If Yes: Increment Level. If No: Decrement Tries -> Check if Tries == 0 (Game Over)]

7. Design Decisions & Rationale

A procedural approach using a 2D list was chosen over a complex Object-Oriented design because the game's scope requires simple state management (level index and tries remaining). This highlights core Python problem-solving concepts like nested loops and list indexing.

8. Implementation Details

The core logic utilizes Python's built-in string methods (`.center()`, `.capitalize()`) and ANSI escape sequences (`[1;3m`) for formatting. The state is maintained using integer variables (`level`, `tries`) that are evaluated in the loop conditions. The game includes three distinct challenges: "Algorithm", "Variable", and "Medicine".

9. Screenshots / Results

[Placeholder: Screenshot of the terminal displaying "LEVEL 1" and the hint for "Algorithm"]

[Placeholder: Screenshot of the "GAME OVER" terminal output after exhausted attempts]

10. Testing Approach

Manual testing was conducted in the terminal to verify the inner loop accurately counts down attempts from 4 to 0, and the outer loop correctly increments from level 0 to 2, concluding with the "YOU WIN" sequence.

11. Challenges Faced

Managing the control flow between the inner guess loop and the outer level loop required careful placement of break statements to ensure a "GAME OVER" scenario completely exited the application rather than just advancing to the next level.

12. Learnings & Key Takeaways

The development process reinforced problem-solving logic regarding state management within nested while loops, and the importance of input normalization (handling capitalization) for reliable conditional evaluations.

13. Future Enhancements

- Extract the procedural code into functions for better modularity.
- Implement a scoring system based on the number of attempts remaining.
- Read levels from an external text or CSV file to separate data from the logic script.

14. References

- VITyarthi - Build Your Own Project Guidelines [cite: 1].
- Python 3 String Methods Documentation.